

PIDAgent: PID Tuning via LLM Agents and RAG

1st Yitao Chen
Ningbo Innovation Center
Zhejiang University
Ningbo, China
22460532@zju.edu.cn

2nd Hongye Su
College of Control Science and Engineering
Zhejiang University
Hangzhou, China
hysu@ipc.zju.edu.cn

3rd Yang Zhu[†]
College of Control Science and Engineering
Zhejiang University
Hangzhou, China
zhuyang88@zju.edu.cn

Abstract—Large language models (LLMs), equipped with advanced capabilities in natural language understanding, generation, and reasoning, are progressively automating routine and repetitive tasks to ease human effort. However, in industrial domains requiring high-precision expertise, LLMs frequently underperform. Focusing on Proportional-Integral-Derivative (PID), this study augments LLMs’ core strengths by incorporating Retrieval-Augmented Generation (RAG), which provides access to control knowledge and tuning experience, thereby enhancing the accuracy and reliability of parameter optimization. External tools are further encapsulated within an Model Context Protocol (MCP) Server, enabling LLMs to execute practical operations beyond theory. This study establishes a three-tiered hierarchy framework, PIDAgent, in which the primary agent conducts task decomposition, subagents implement domain-specific functions, and MCP tools execute practical operations, thus enabling end-to-end autonomous tuning. Simulation results show that the proposed PIDAgent framework meets target performance metrics with a 95% success rate, demonstrating its practicality for industrial PID controllers.

Index Terms—LLMs, Agent, RAG, MCP Tools, PID Tuning

I. INTRODUCTION

Large language models (LLMs) have emerged as transformative innovations in artificial intelligence, leveraging advanced reasoning and cognitive capabilities to propel progress across diverse domains. LLMs demonstrate exceptional proficiency in handling complex natural language tasks, including summarization, dialogue generation, and creative content synthesis. Moreover, their human-like reasoning and contextual comprehension enable versatile problem-solving in areas demanding nuanced analytical insight, such as automated programming and autonomous control systems [1]–[3]. Illustrating this progress, recent state-of-the-art models such as OpenAI’s GPT-5 and DeepSeek-R1 exemplify the next stage of LLM evolution, featuring enhanced generalization, multimodal integration, and adaptive task handling that signal continued advancement toward more unified and capable AI systems [4].

Nevertheless, LLMs exhibit critical vulnerabilities in dynamic, domain-specific scenarios. Stripped of real-time data access and external knowledge, their reliance on probabilistic

generation frequently induces hallucinations—producing plausible but factually incorrect outputs. Moreover, LLMs lack direct execution capabilities. Although they can formulate conceptual strategies, their inability to autonomously interface with external APIs or physical infrastructures without supporting frameworks severely limits their practical utility in interactive tasks [5].

To suppress inaccuracies in model outputs, methodologies such as Retrieval-Augmented Generation (RAG), first proposed by Lewis et al. at Facebook AI research, integrate on-the-fly access to relevant external knowledge base during inference, grounding responses in verified sources by incorporating retrieved information directly into prompts rather than relying on costly model fine-tuning [6], [7]. While RAG enhances factual reliability, it does not equip LLMs with the capability for autonomous execution. To address this limitation, the Model Context Protocol (MCP), an open interoperability standard introduced by Anthropic, enables secure and bidirectional communication between LLMs and diverse data streams or functional modules, supporting structured invocations and real-time interactions that transform static models into dynamic, responsive systems [8]. Building upon these advances, agent-oriented paradigms such as ReAct further extend LLMs’ capabilities by integrating reasoning and action within interactive environments. This fusion of linguistic cognition and environmental feedback allows agents to iteratively refine their strategies and exhibit greater robustness in handling unexpected scenarios [9].

This study focuses on Proportional-Integral-Derivative (PID) tuning, which plays a crucial role in optimizing PID controllers, widely used in manufacturing automation for their simplicity and robustness. However, in industrial applications, conventional PID tuning typically depends heavily on manual adjustments and domain expertise, resulting in a process that is labor-intensive, difficult to standardize, and highly sensitive to operator experience. Traditional calibration strategies, such as the Ziegler–Nichols and Cohen–Coon methods, rely on heuristic rules and iterative experimentation to balance stability, performance, and robustness [10], [11]. Advanced tuning variants, including model-based calibration and heuristic algorithms, enable dynamic parameter adjustment. Yet, their reliance on expert supervision, poor real-time flexibility, and high computational cost under nonlinearities severely limit their scalability. Consequently, the increasing complexity

This work was partially supported by Zhejiang R&D Program (Grant No. 2025C01022), National Natural Science Foundation of China (Grant No. 62303410), Zhejiang Provincial Natural Science Foundation of China (Grant No. LQ23F030014), and Open Research Project of State Key Laboratory of Industrial Control Technology, China (Grant No. ICT2025B78, ICT2025B09).

[†]Corresponding Author: Yang Zhu

of modern industrial processes necessitates intelligent, data-driven tuning strategies that offer autonomous adaptation and robust generalization across diverse system dynamics.

To better facilitate the application of LLMs in industrial scenarios, some integrations of LLMs with the control field have already been implemented. Frameworks such as ControlAgent orchestrate multi-agent collaborations to streamline control configuration by combining the reasoning capabilities of LLMs with domain-specific expertise. Another paradigm, the LLM-Agent-Controller, establishes a flexible ensemble of agents to support controller design, simulation, and stability analysis. In addition, LLM-based PID controller optimization directly employs prompt engineering to achieve automatic PID tuning [12]–[14]. Nevertheless, few studies have investigated frameworks that combine LLM agents with RAG for PID tuning.

To achieve accurate automatic PID tuning, we propose PIDAgent, an innovative hierarchical framework that augments LLM reasoning with two key refinements. First, Retrieval-Augmented Generation (RAG) integrates domain-specific tuning expertise to ground outputs and mitigate hallucinations. Second, a Model Context Protocol (MCP) server integrates external tools (e.g., MATLAB simulations and performance calculators), enabling the LLMs to interact with simulated environments rather than relying on abstract generation. The framework structurally comprises a primary agent for global planning, specialized subagents for granular execution, and MCP tools for practical validation.

It is important to clarify that PIDAgent is not designed to replace high-frequency real-time controllers. Due to the inherent latency of LLMs, this framework is specifically targeted at scenarios such as offline auto-tuning, slow-dynamic process control, and supervisory parameter scheduling, where computational overhead is well justified by the reduction in expert labor costs.

The contributions of this paper can be summarized as follows:

- **Self-Updating Expert Knowledge Base:** Within PID tuning, RAG mitigates hallucination by grounding responses in expert knowledge base. The resulting tuning parameters can then be feedback into the knowledge base, enabling continuous refinement and improved accuracy over time.
- **End-to-End Automation:** The framework delivers end-to-end solutions, freeing users from the need for domain expertise. Through the coordination of the primary agent, subagents, and MCP tools, it generates PID parameters that meet the desired performance metrics.
- **Generalized Adaptive Optimization:** The framework applies not only to model-based systems but also to complex or model-free scenarios. By autonomously selecting tools, executing actions, observing outcomes, and iteratively refining decisions, it derives optimal parameters.

II. METHODS

A. Problem Formulation

In this study, a PID controller is designed using LLMs for both model-based and model-free systems. As shown in (1), based on the user prompt q , PIDAgent initializes and adjusts the proportional gain k_c , integral time t_i , and derivative time t_d so that the performance metrics, overshoot o , settling time s , and steady-state error ss , satisfy the target t .

$$(k_c, t_d, t_i, o, s, ss) = \text{PIDAgent}(q), \quad \{o, s, ss\} \subseteq t \quad (1)$$

B. Architecture of PIDAgent

As shown in Fig. 1, PIDAgent is an end-to-end framework composed of a primary agent, subagents, and MCP tools. The primary agent formulates decision-making plans for problems and autonomously deploys subagents. The agent uses the React framework, which combines introspective analysis with practical interventions in AI systems. The LLM first orchestrates subagents according to the user prompt, then observes their execution results, and finally reflects on the process before proceeding to the next step. This allows agents to seamlessly alternate between generating insightful analysis and performing context-specific actions, enabling fluid adaptation to evolving conditions.

To ensure reliable multi-agent execution, prompts are standardized across all agents. Explicit role definitions assign specific tasks (e.g., designating the primary agent as the global router and evaluator, and subagents as specialized PID tuning experts), clear context formatting uses delimiters to separate instructions from execution history, and strict output constraints mandate structured responses like JSON. This structured design guarantees deterministic parsing and seamless communication with MCP tools.

The framework deploys three specialized subagents: Model-Free Initialization, Model-Based Initialization, and Knowledge-Guided Tuning. Their interaction with external environments is governed by the Model Context Protocol (MCP), a secure and standardized communication layer. Functioning autonomously as MCP clients, these subagents parse tasks, query registered MCP servers, and directly invoke required tools (e.g., MATLAB simulations or performance evaluators). The server executes the requests in a sandboxed environment and returns structured outputs. By encapsulating internal logic and tool-access within each subagent, the framework eliminates the need for centralized low-level tool orchestration, ensuring modular and efficient execution.

The MCP tools include RAG tools, which have the functions of retrieving the initial knowledge base and retrieving the parameter adjustment rule knowledge base, and adding the tuned knowledge to the initial knowledge base. Execution tools include MATLAB simulation tools. Evaluation tools include tools for calculating performance metrics, visualization, and verifying whether the performance metrics meet specified targets. These tools are encapsulated through an MCP Server, enabling the LLM to autonomously select and invoke the appropriate tools as needed.

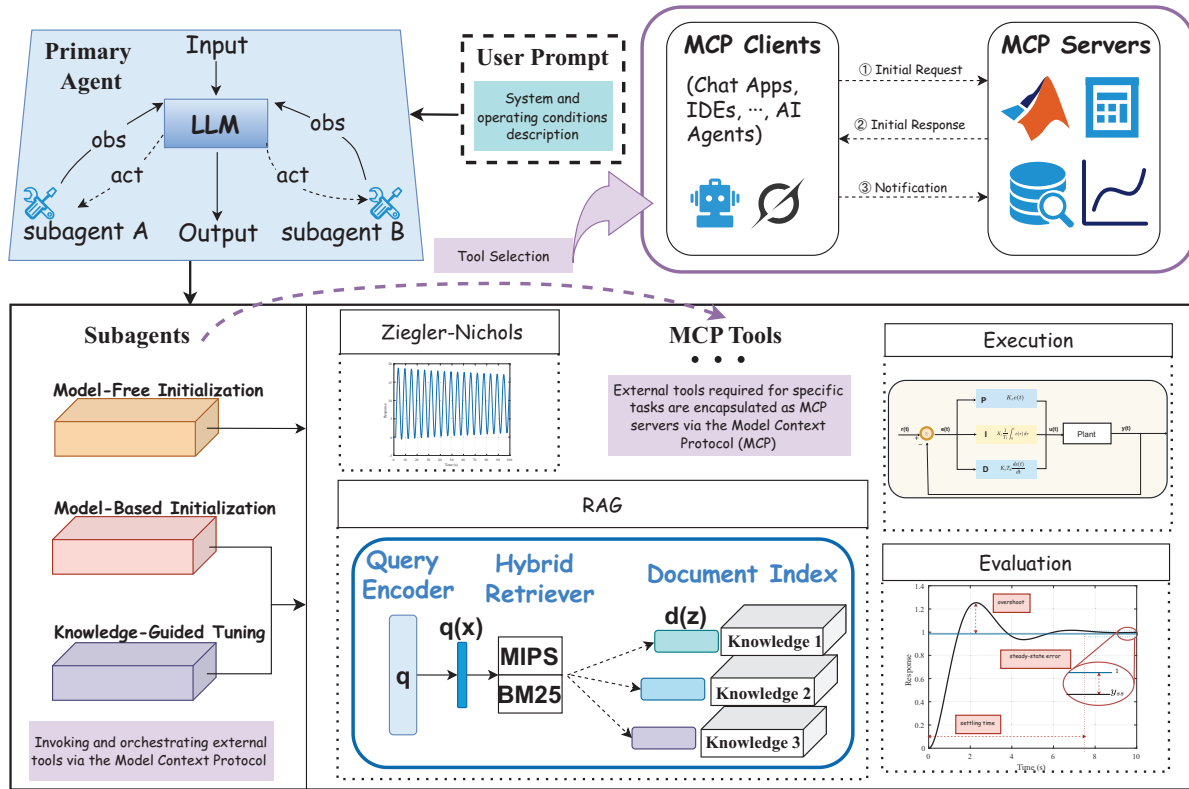


Fig. 1: An overview of PIDAgent

C. PIDAgent workflow

Based on the user prompt, the primary agent autonomously determines whether the model is specific, then enters the corresponding initialization subagent. It obtains the initial values of k_c , t_i , and t_d , and subsequently evaluates whether the performance metrics meet the targets. If so, it returns the current parameters. If not, it enters the Knowledge-Guided Tuning, continuing the process until the targets are met or the maximum number of iterations is reached.

For input with a specific model, refer to pseudocode in Algorithm 1. The primary agent first determines whether the input is model-based. If so, the Model-Based Initialization subagent is invoked, where the initial knowledge retriever tool from RAG tools queries the external knowledge base for information related to the current model and incorporates it into the prompt.

The initial knowledge retriever tool begins by loading the external knowledge base and preprocessing raw textual data through a chunking strategy that segments documents into semantically coherent units. Each chunk preserves key contextual elements, such as model formulas and descriptive metadata, ensuring interpretability throughout the retrieval process. The resulting segments are then transformed into dense

vector representations using advanced embedding models, such as OpenAI's text-embedding-3-large, and stored within a vector index for efficient retrieval. As illustrated in Fig. 1, the RAG tools process the user prompt by first passing it through the Query Encoder, where it is converted into a vector representation and then handled by a hybrid retriever that combines BM25 and Maximum Inner Product Search (MIPS). BM25 captures term-level lexical relevance, ensuring keyword precision, while MIPS measures semantic proximity in the embedding space, enabling contextual alignment. This dual mechanism balances exact-match accuracy with semantic flexibility, yielding more robust and contextually grounded retrieval results—particularly effective in mitigating factual drift and enhancing response reliability in RAG-based systems. Finally, the retriever identifies the top- k most relevant documents from the vector database, extracts their chunked natural-language contents, and concatenates them with the original query to form the final prompt input.

The initial knowledge base utilized in this framework draws from Aidan O'Dwyer's Handbook of PI and PID Controller Tuning Rules, a seminal reference that consolidates decades of control engineering advancements [15]. This book serves as an essential guide for practitioners and researchers in process control, offering a standardized compilation of tuning

Algorithm 1 PIDAgent's Model-Based PID Tuning

Parameters Definition: c : Related knowledge found in the external knowledge, k : Number of iterations.

Input: q

Output: k_{c2}, t_{i2}, t_{d2}

```
if Model-Based  $\leftarrow$  PrimaryAgent( $q$ ) then
     $c_1 \leftarrow$  InitialRetriever( $q$ )
     $k_{c1}, t_{i1}, t_{d1} \leftarrow$  Subagent( $q, c_1$ )
     $o_1, s_1, ss_1 \leftarrow$  SimuAndVisual( $q, k_{c1}, t_{i1}, t_{d1}$ )
    if True  $\leftarrow$  IsMeetTargets( $o_1, s_1, ss_1, t$ ) then
        break
    else
        for  $k = 0$  to  $iter_{max}$  do
             $c_2 \leftarrow$  TuningRuleRetriever( $q$ )
             $k_{c2}, t_{i2}, t_{d2} \leftarrow$  Subagent( $q, c_2$ )
             $o_2, s_2, ss_2 \leftarrow$  SimuAndVisual( $q, k_{c2}, t_{i2}, t_{d2}$ )
            if True  $\leftarrow$  IsMeetTargets( $o_2, s_2, ss_2, t$ ) then
                break
            end if
        end for
    end if
end if
```

methodologies to optimize PI and PID controllers across diverse industrial applications. It emphasizes practical tuning strategies that enhance system stability, response speed, and robustness against disturbances, making it invaluable for scenarios involving time delays or complex dynamics.

Afterward, the LLM generates the initial PID parameters, which immediately undergo a hard-coded boundary check (e.g., $k_c > 0$, $t_i > 0$, $t_d \geq 0$) to ensure physical feasibility and intercept hallucinatory extreme values. The LLM then calls the execution and evaluation tools to simulate the system, calculate performance metrics, visualize results, and compare them with the targets. If the targets are met, the process returns directly; if not, it triggers the Knowledge-Guided Tuning subagent. First, the tuning rule knowledge retriever tool from the RAG tools is called to find the corresponding parameter adjustment rules. Subsequently, the same process as the Model-Based Initialization subagent is executed. This iterative tuning continues through the Knowledge-Guided Tuning subagent until the targets are met or the maximum number of iterations is reached.

For complex models or systems that are difficult to model, refer to pseudocode in Algorithm 2. The primary agent first determines whether the input is model-free based on the input. If so, it calls the Model-Free Initialization subagent. According to the Ziegler-Nichols method, t_i and t_d are not considered for the time being, and k_c is gradually increased until the simulation results show equal amplitude oscillations. The current k_u and t_u are recorded, and the PID parameters are obtained according to the empirical formula. Then, the execution and evaluation tools are called to simulate. The subsequent process is consistent with the Model-Based PID tuning.

Algorithm 2 PIDAgent's Model-Free PID Tuning

Parameters Definition: k_u : The proportional gain at which the system output exhibits sustained oscillations, t_u : The period of oscillations observed when the controller gain is set to the ultimate gain.

Input: q

Output: k_{c2}, t_{i2}, t_{d2}

```
if Model-Free  $\leftarrow$  PrimaryAgent( $q$ ) then
     $k_{u1}, t_{u1} \leftarrow$  Ziegler-Nichols( $q$ )
     $k_{c1}, t_{i1}, t_{d1} \leftarrow$  Subagent( $k_{u1}, t_{u1}$ )
     $o_1, s_1, ss_1 \leftarrow$  SimuAndVisual( $q, k_{c1}, t_{i1}, t_{d1}$ )
     $o_2, s_2, ss_2 \leftarrow$  ValiAndTuning( $o_1, s_1, ss_1, t, p$ )
end if
```

III. EXPERIMENTS

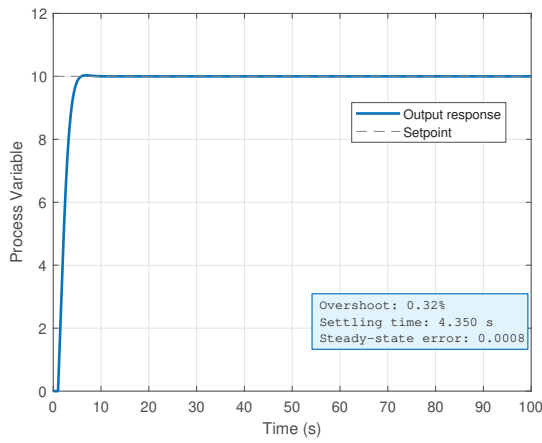
This section presents the simulation results for the PIDAgent, highlighting its feasibility and versatility. The framework employs a heterogeneous multi-layered architecture—comprising a primary agent, subagents, and MCP tools—and adopts a task-specific model routing strategy to maximize performance.

Specifically, the DeepSeek-R1 model is exclusively deployed in the Model-Based Initialization subagent. Its exceptional capabilities in multi-step reasoning and complex mathematical abstraction ensure high accuracy during rigorous modeling tasks. Conversely, the Qwen3-Max model powers the primary agent and the remaining subagents. With its rapid inference speed, superior natural language comprehension, and robust instruction execution, Qwen3-Max is optimally suited for global coordination and multi-task scheduling. By implementing this "fit-for-purpose" model deployment strategy, PIDAgent fully leverages the complementary strengths of different LLMs, achieving highly efficient agent collaboration and intelligent decision-making.

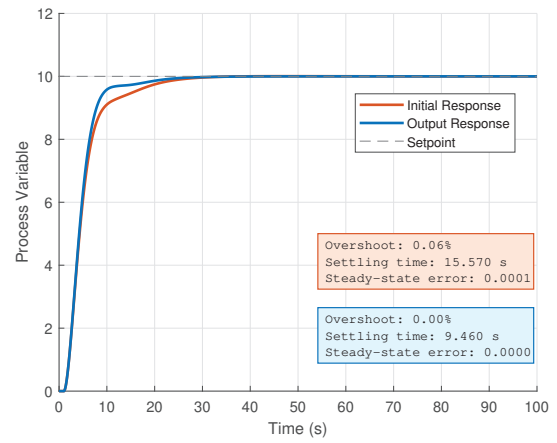
A. Feasibility Study under Model-Based Conditions

In certain industrial scenarios, where systems exhibit well-defined behaviors and characteristics, a specific model can be built based on these unique conditions. In such cases, PID tuning typically begins with applying empirical formulas derived from established control theories, followed by manual adjustments based on the system's response curve and performance metrics. This process involves tuning the proportional gain, integral time, and derivative time by analyzing metrics such as overshoot, settling time, and steady-state error. While effective, this method is time-consuming and heavily reliant on the expertise of operators. The introduction of LLMs to replace the manually tuned PIDAgent automates this process.

To verify the feasibility of PIDAgent, this study conducts simulations on three representative system models with increasing complexity: the Second-Order System Plus Dead Time (SOSPD), the Third-Order System Plus Dead Time (TOSPD), and the Fourth-Order System Plus Dead Time (FOSPD).



(a) Second-Order Plus Dead Time



(b) Third-Order System Plus Dead Time

Fig. 2: Output of SOSPD and TOSPD

$$G(s) = \frac{Ke^{-sT}}{T_m^2 s^2 + 2\xi_m T_m s + 1} \quad (2)$$

Equation (2) represents the SOSPD model, which is widely used to characterize industrial processes exhibiting inertial dynamics and time delays. Where K denotes the system gain, T_m is the time constant, ξ_m is the damping ratio, and T represents the time delay. This type of system is common in various engineering fields where the process output cannot respond immediately to input changes due to physical limitations or transmission delays. It is applicable to a variety of systems, including thermal processes, fluid flow, mechanical servo, and chemical reactions. Taking an underdamped system as an example, after entering the specific model and parameters via a user prompt, PIDAgent generates PID parameters that meet the targets. Fig. 2a shows the specific step response and the calculated performance metrics.

$$G(s) = \frac{Ke^{-sT}}{(1 + sT_{m1})(1 + sT_{m2})(1 + sT_{m3})} \quad (3)$$

Equation (3) denotes the TOSPD model. T_{m1} , T_{m2} , and T_{m3} are the system's time constants, describing the inertia characteristics of different dynamic links. Compared to the second-order model, the third-order model more accurately reflects the dynamic behavior of multi-stage processes (such as multiple tanks, series heat exchangers, or multi-inertial mechanical systems), particularly by demonstrating more pronounced delays and multi-stage transitions in the output response. Fig. 2b shows the step response and performance metrics calculated by PIDAgent for this model.

$$G(s) = \frac{Ke^{-sT}}{(1 + sT_{m1})^4} \quad (4)$$

Equation (4) corresponds to the FOSPD model. T_{m1} represents the time constant of each inertial link, and the series con-

nection of four identical time constants reflects the dynamic characteristics of the system's multi-level inertia accumulation. The PIDAgent processing results are shown in Fig. 3a.

B. Feasibility Study under Model-Free Conditions

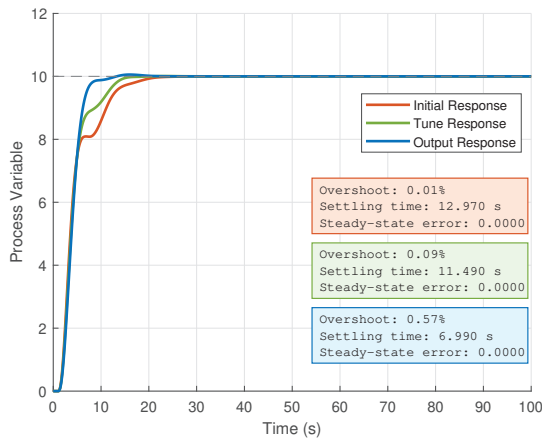
In industrial scenarios, many processes exhibit complex dynamic characteristics or are difficult to model accurately, making empirical formulas based on model parameters inapplicable. Therefore, this study used the Ziegler-Nichols tuning method to experimentally determine the system's limiting gain K_u and oscillation period T_u , from which the initial PID parameters were calculated. If performance metrics do not meet targets, the Knowledge-Guided Tuning subagent is used for optimization. Fig. 3b shows the PIDAgent's output under model-free conditions.

C. Generalization Evaluation

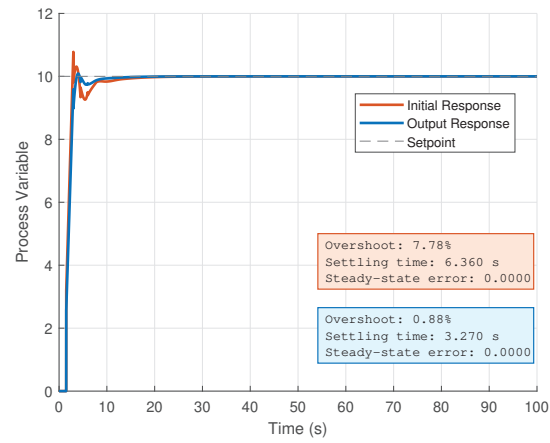
The feasibility analysis above focuses on validating PIDAgent for representative models, but testing its generalization across different parameter types is equally important. Next, experiments were conducted on 20 different parameter models for SOSPD, TOSPD, and FOSPD to evaluate the correctness and efficiency of the PIDAgent. Table I presents the experimental results, with an average success rate of 95% over all model types. *per* indicates the probability that PIDAgent's generated PID parameters met the target performance in 20 experiments, while *n* denotes the mean number of parameter adjustments (including initialization) required for success.

TABLE I: Generalization Evaluation

Model	<i>per</i> [%]	<i>n</i>
Second-Order Plus Dead Time	100	1.2
Third-Order System Plus Dead Time	100	2.1
Fourth-Order System Plus Dead Time	85	2.9



(a) Fourth-Order System Plus Dead Time



(b) Model-Free

Fig. 3: Output of FOSPD and Model-Free

D. Ablation Study on Retrieval-Augmented Generation

To mitigate the hallucination problem in large models, this study applies RAG technology to PID tuning. Table II presents an ablation experiment with RAG. As shown, removing RAG and allowing the LLM to adjust autonomously without external knowledge or human expertise causes a significant drop in accuracy. This highlights the critical role of an external knowledge base in reducing hallucinations.

TABLE II: Ablation Study of RAG

Method	SOSPD	TOSPD	FOSPD
PIDAgent*	20	5	0
+ RAG	100	100	85
<i>w.r.t PIDAgent*</i>	+80	+95	+85

* denotes PIDAgent without RAG.

IV. CONCLUSION

This paper addresses the limitation of traditional PID tuning, which requires extensive expertise and manual intervention, by enabling LLMs to automate the tuning of PID controllers—widely used in industrial applications. PIDAgent, an end-to-end framework, allows users to obtain optimal PID parameters that achieve target performance metrics by simply entering a user prompt, without any knowledge of professional PID controllers or relevant PID tuning experience. For inputs with specific models, PIDAgent, with the help of the primary, subagents, and MCP tools, can obtain the initialization parameters of the corresponding model and perform tuning based on an external parameter adjustment rule knowledge base. For complex models or inputs that are difficult to model, LLMs can independently use the Z-N experimental method to obtain the usable initialization parameters and perform tuning. Matlab simulation experiments show that PIDAgent not only achieves high accuracy, but also has strong generalization and extensibility, making it suitable for PID tuning.

REFERENCES

- [1] X. Hou, Y. Zhao, and Y. Liu, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, 2024.
- [2] F. Xu, Q. Hao, and C. Shao, "Toward large reasoning models: A survey of reinforced reasoning with large language models," *Patterns*, vol. 6, no. 10, 2025.
- [3] W. X. Zhao et al., "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
- [4] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, and X. Bi, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.
- [5] L. Huang, W. Yu, W. Ma, et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, 2025.
- [6] A. Madaan, N. Tandon, P. Gupta, et al., "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534–46594, 2023.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, and T. Rocktäschel, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [8] X. Hou, Y. Zhao, and S. Wang, "Model Context Protocol (MCP): Landscape, security threats, and future research directions," *arXiv preprint arXiv:2503.23278*, 2025.
- [9] S. Yao, J. Zhao, and D. Yu, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Representations (ICLR)*, 2022.
- [10] J. G. Ziegler and N. B. Nichols, "Optimum settings for automatic controllers," *Trans. Amer. Soc. Mech. Eng.*, vol. 64, no. 8, pp. 759–765, 1942.
- [11] G. H. Cohen and G. A. Coon, "Theoretical consideration of retarded control," *Trans. Amer. Soc. Mech. Eng.*, vol. 75, no. 5, pp. 827–834, 1953.
- [12] X. Guo, K. Keivan, and U. Syed, "ControlAgent: Automating control system design via novel integration of LLM agents and domain expertise," *arXiv preprint arXiv:2410.19811*, 2024.
- [13] R. Zahedifar, S. A. Mirghasemi, and M. S. Baghshah, "LLM-Agent-Controller: A universal multi-agent large language model system as a control engineer," *arXiv preprint arXiv:2505.19567*, 2025.
- [14] I. Kamenko, S. Ilic, and V. Congradac, "LLM-based PID controller optimization," in *Proc. 2025 10th Int. Conf. Smart and Sustainable Technologies (SpliTech)*, IEEE, pp. 1–6, 2025.
- [15] A. O'Dwyer, *Handbook of PI and PID Controller Tuning Rules*. Singapore: World Scientific, 2009.